Homework 10

1(a)

Seed: **43210**. RLT version: **6.1.1**; reinforcement turned off.

Squared bias is calculated against $f(\mathbf{x}) = x_1 + x_3 + x_5$. Bias squared and sample variance are estimated across 100 repetitions at each fixed test point, then averaged over the 200 test points.

```
# Run once if RLT is not installed:
# install.packages("RLT", repos = c("https://teazrq.r-universe.dev",
#                                   "https://cloud.r-project.org"))
library(RLT)
seed <- 43210L
set.seed(seed)
B <- 100L
n <- 200L
n_test <- 200L
Sigma <- matrix(0.3, 5, 5)
diag(Sigma) <- 1
L <- chol(Sigma)
generate_x <- function(n) matrix(rnorm(n * 5), n, 5) %*% L
x_test <- generate_x(n_test)
f_test <- rowSums(x_test[, c(1, 3, 5)])
settings <- unique(rbind(
  data.frame(nmin = c(1, 5, 10, 20, 30), mtry = 3),
  data.frame(nmin = 10, mtry = 1:5)
))
# Generate all training sets before forest fitting to isolate data randomness.
training <- lapply(seq_len(B), function(b) {
  x <- generate_x(n)
  list(x = x, y = rowSums(x[, c(1, 3, 5)]) + rnorm(n))
})
forest_seeds <- matrix(sample.int(.Machine$integer.max, B * nrow(settings)),
                       B, nrow(settings))
predictions <- array(NA_real_, c(n_test, B, nrow(settings)))
for (b in seq_len(B)) {
  for (s in seq_len(nrow(settings))) {
    fit <- RLT(training[[b]]$x, training[[b]]$y,
               model = "regression", ntrees = 500,
               nmin = settings$nmin[s], mtry = settings$mtry[s],
               resample.replace = TRUE, resample.prob = 1,
               reinforcement = FALSE, seed = forest_seeds[b, s])
    predictions[, b, s] <- as.numeric(predict(fit, x_test)$Prediction)
  }
  if (b %% 10 == 0) message("Completed repetition ", b)
}
results <- settings
results$bias_squared <- vapply(seq_len(nrow(settings)), function(s) {
```

```
  mean((rowMeans(predictions[, , s]) - f_test)^2)
}, numeric(1))
results$variance <- vapply(seq_len(nrow(settings)), function(s) {
  mean(apply(predictions[, , s], 1, var))
}, numeric(1))
results$total <- results$bias_squared + results$variance
```

```
node_results <- results[results$mtry == 3, ]
node_results <- node_results[order(node_results$nmin), ]
mtry_results <- results[results$nmin == 10, ]
mtry_results <- mtry_results[order(mtry_results$mtry), ]
knitr::kable(results, digits = 4,
             col.names = c("Node size", "mtry", "Squared bias", "Variance",
                           "Squared bias + variance"))
```

	Node size	mtry	Squared bias	Variance	Squared bias + variance
1	1	3	0.2302	0.2021	0.4322
2	5	3	0.2355	0.1972	0.4326
3	10	3	0.2545	0.1841	0.4385
4	20	3	0.3303	0.1646	0.4949
5	30	3	0.4343	0.1597	0.5940
6	10	1	0.6566	0.1143	0.7709
7	10	2	0.3360	0.1487	0.4847
9	10	4	0.2312	0.2104	0.4416
10	10	5	0.2205	0.2306	0.4511

```
plot_tradeoff <- function(x, tab, xlab) {
  matplot(x, cbind(tab$bias_squared, tab$variance),
          type = "b", pch = c(16, 17), lty = c(1, 2),
          col = c("#0072B2", "#D55E00"), lwd = 2,
          xlab = xlab, ylab = "Average over 200 test points",
          ylim = c(0, 1.12 * max(tab$bias_squared, tab$variance)),
          xaxt = "n")
  axis(1, at = x)
  legend("topleft", c("Squared bias", "Variance"),
         col = c("#0072B2", "#D55E00"), pch = c(16, 17),
         lty = c(1, 2), lwd = 2, bty = "n")
}
```

```
plot_tradeoff(node_results$nmin, node_results, "Node size (nmin)")
```

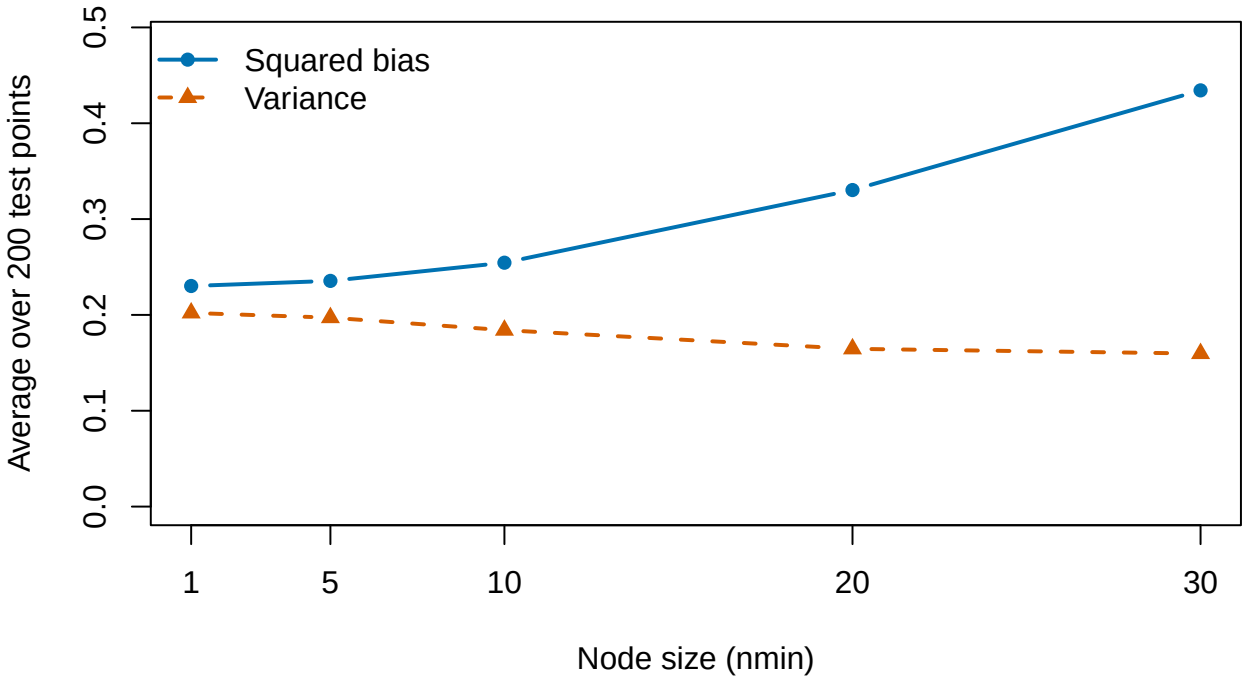


Figure 1: Squared bias and variance as node size changes, with mtry fixed at 3.

```
plot_tradeoff(mtry_results$mtry, mtry_results, "Candidate predictors (mtry)")
```

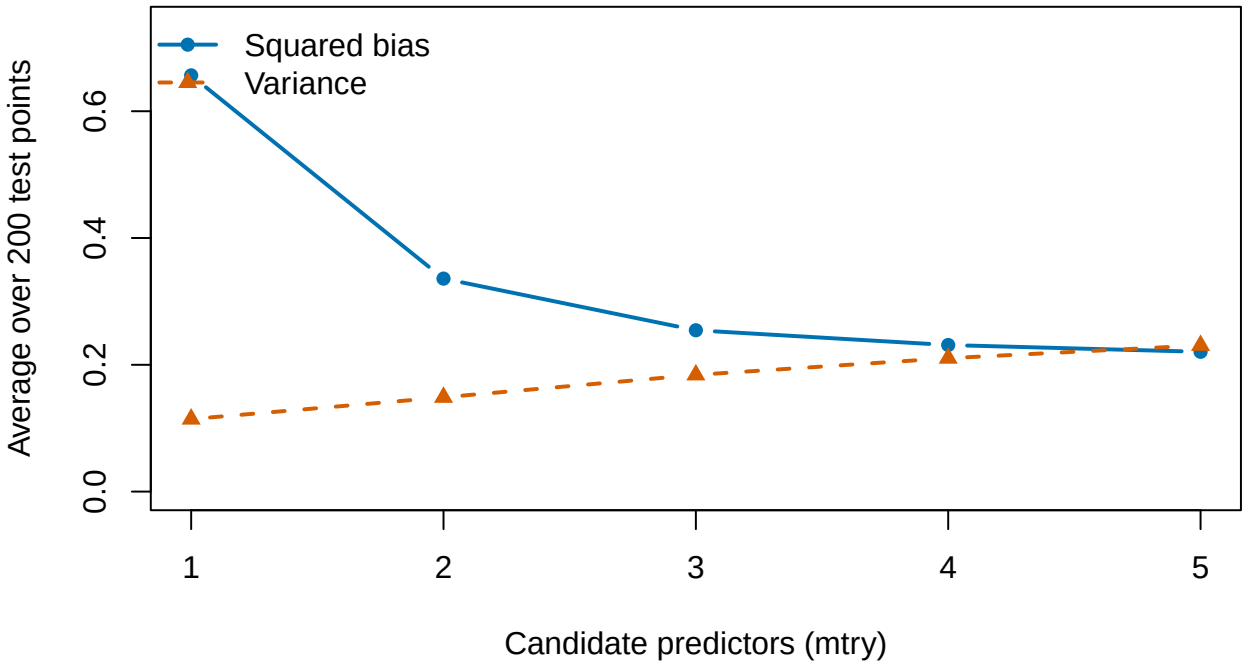


Figure 2: Squared bias and variance as mtry changes, with node size fixed at 10.

1(b)

Node size. Increasing node size increases squared bias and decreases variance. A node size of 30 stops splitting earlier, producing coarser predictions that underfit the regression function. From node size 1 to 30, squared bias rises from 0.2302 to 0.4343, while variance falls from 0.2021 to 0.1597. Node sizes 1 and 5 have nearly identical squared bias plus variance (0.4322 and 0.4326).

Candidate predictors. A larger `mtry` increases the chance of considering informative predictors at each split, reducing bias. However, it can make trees more similar, increasing forest variance, and requires more computation per split. Here, increasing `mtry` from 1 to 5 reduces squared bias from 0.6566 to 0.2205 but increases variance from 0.1143 to 0.2306. **mtry = 3** gives the lowest squared bias plus variance among the tested values.

Sampling 100 observations per tree. I would expect the preferred node size to shift toward smaller values because the same stopping threshold represents a larger fraction of a smaller bootstrap sample. Smaller nodes allow more splitting and can offset the resulting bias. However, fewer observations also make predictions less stable, which can favor larger nodes to control variance. The direction of the shift is therefore not guaranteed.

2(a)

Seed: **43220**. PCA uses the first 20 components fitted to centered, unscaled training pixels. All classification forests use 1,000 trees, full-size bootstrap samples, and no reinforcement.

```
library(RLT)
seed_q2 <- 43220L
set.seed(seed_q2)
mnist <- read.csv(gzfile("data/mnist-first2000.csv.gz"))
```

```r
stopifnot(nrow(mnist) == 2000, ncol(mnist) == 785, "Digit" %in% names(mnist))
digits <- c(1, 6, 7)
train_rows <- which(seq_len(nrow(mnist)) <= 1000 & mnist$Digit %in% digits)
pixel_names <- setdiff(names(mnist), "Digit")
y_train <- factor(mnist$Digit[train_rows], levels = digits)
pca <- prcomp(mnist[train_rows, pixel_names], center = TRUE, scale. = FALSE,
              rank. = 20)
x_train <- pca$x[, 1:20]
colnames(x_train) <- paste0("PC", 1:20)
variance_explained <- sum(pca$sdev[1:20]^2) / sum(pca$sdev^2)
# Omit mtry and nmin to use RLT's actual defaults.
default_fit <- RLT(x_train, y_train, model = "classification", ntrees = 1000,
                   resample.replace = TRUE, resample.prob = 1,
                   reinforcement = FALSE, seed = seed_q2)
default_error <- mean(default_fit$Prediction != y_train)
stopifnot(isTRUE(all.equal(default_error, as.numeric(default_fit$Error))))
# Specify the grid and tie-break rule before observing its errors.
grid <- expand.grid(mtry = c(1, 2, 4, 6, 10, 15, 20),
                    nmin = c(1, 5, 10, 20, 30))
grid$oob_errors <- NA_integer_
grid$oob_rate <- NA_real_
fits <- vector("list", nrow(grid))
for (g in seq_len(nrow(grid))) {
  # Permutation importance is computed for each fit so the selected
  # forest can be retained exactly, without refitting after selection.
  fits[[g]] <- RLT(x_train, y_train, model = "classification", ntrees = 1000,
                   mtry = grid$mtry[g], nmin = grid$nmin[g],
                   resample.replace = TRUE, resample.prob = 1,
                   reinforcement = FALSE, importance = "permute",
                   seed = seed_q2)
  grid$oob_errors[g] <- sum(fits[[g]]$Prediction != y_train)
  grid$oob_rate[g] <- mean(fits[[g]]$Prediction != y_train)
}
# Ties: prefer larger nodes, then smaller mtry.
best <- order(grid$oob_rate, -grid$nmin, grid$mtry)[1]
selected <- grid[best, ]
selected_fit <- fits[[best]]
importance_table <- data.frame(PC = colnames(x_train),
                               OOB_permutation_importance = as.numeric(selected_fit$VarImp))
importance_table <- importance_table[order(-importance_table$OOB_permutation_importance), ]
# Only after selection: filter and assess the held-out observations.
test_rows <- which(seq_len(nrow(mnist)) > 1000 & mnist$Digit %in% digits)
x_test_mnist <- predict(pca, newdata = mnist[test_rows, pixel_names])[, 1:20]
colnames(x_test_mnist) <- colnames(x_train)
y_test <- factor(mnist$Digit[test_rows], levels = digits)
test_prediction <- predict(selected_fit, x_test_mnist)$Prediction
confusion <- table(Actual = y_test, Predicted = test_prediction)
test_errors <- sum(test_prediction != y_test)
test_error <- mean(test_prediction != y_test)
stopifnot(sum(confusion) == length(y_test), all(is.finite(x_train)),
          all(is.finite(x_test_mnist)), nrow(importance_table) == 20)
```

The default settings are **mtry = 10** and **node size = 5**. The OOB misclassification rate is **4.28%** (14 / 327 observations).

2(b)

The grid uses `mtry` = 1, 2, 4, 6, 10, 15, 20 and node size = 1, 5, 10, 20, 30. Selection minimizes OOB error; ties favor larger node size, then smaller `mtry`.

```
knitr::kable(transform(grid, oob_rate = 100 * oob_rate), digits = 2,
             row.names = FALSE,
             col.names = c("mtry", "Node size", "OOB errors", "OOB error (%)"))
```

mtry	Node size	OOB errors	OOB error (%)
1	1	6	1.83
2	1	10	3.06
4	1	12	3.67
6	1	12	3.67
10	1	13	3.98
15	1	15	4.59
20	1	16	4.89
1	5	8	2.45
2	5	13	3.98
4	5	12	3.67
6	5	12	3.67
10	5	14	4.28
15	5	14	4.28
20	5	16	4.89
1	10	6	1.83
2	10	12	3.67
4	10	12	3.67
6	10	12	3.67
10	10	14	4.28
15	10	14	4.28
20	10	17	5.20
1	20	8	2.45
2	20	11	3.36
4	20	12	3.67
6	20	12	3.67
10	20	14	4.28
15	20	14	4.28
20	20	17	5.20
1	30	10	3.06
2	30	12	3.67
4	30	12	3.67
6	30	12	3.67
10	30	14	4.28
15	30	14	4.28
20	30	17	5.20

The selected setting is **mtry = 1, node size = 10**, with **1.83% OOB error** (6 / 327). Node sizes 1 and 10 tie at six errors when `mtry = 1`, so the tie-break selects 10. Smaller `mtry` performs better in this grid, possibly because it produces more diverse trees. Small OOB differences should not be overinterpreted.

OOB permutation importance for all 20 PCs is reported below.

```
knitr::kable(importance_table, digits = 5, row.names = FALSE,
             col.names = c("Principal component", "OOB permutation importance"))
```

Principal component	OOB permutation importance
PC2	0.12252
PC1	0.09283
PC3	0.03224
PC6	0.02920
PC5	0.02751
PC11	0.02143
PC7	0.02013
PC18	0.01871
PC14	0.01751
PC4	0.01737
PC12	0.01687
PC16	0.01661
PC10	0.01546
PC13	0.01385
PC20	0.01347
PC9	0.01230
PC17	0.01156
PC19	0.01148
PC8	0.01129
PC15	0.01067

```r
par(mar = c(4.5, 4, 1, 1))
barplot(rev(importance_table$OOB_permutation_importance),
        names.arg = rev(importance_table$PC), horiz = TRUE, las = 1,
        col = "#0072B2", border = NA,
        xlab = "OOB permutation importance", cex.names = 0.8,
        xlim = c(0, 1.05 * max(importance_table$OOB_permutation_importance)))
abline(v = 0, col = "gray40")
```

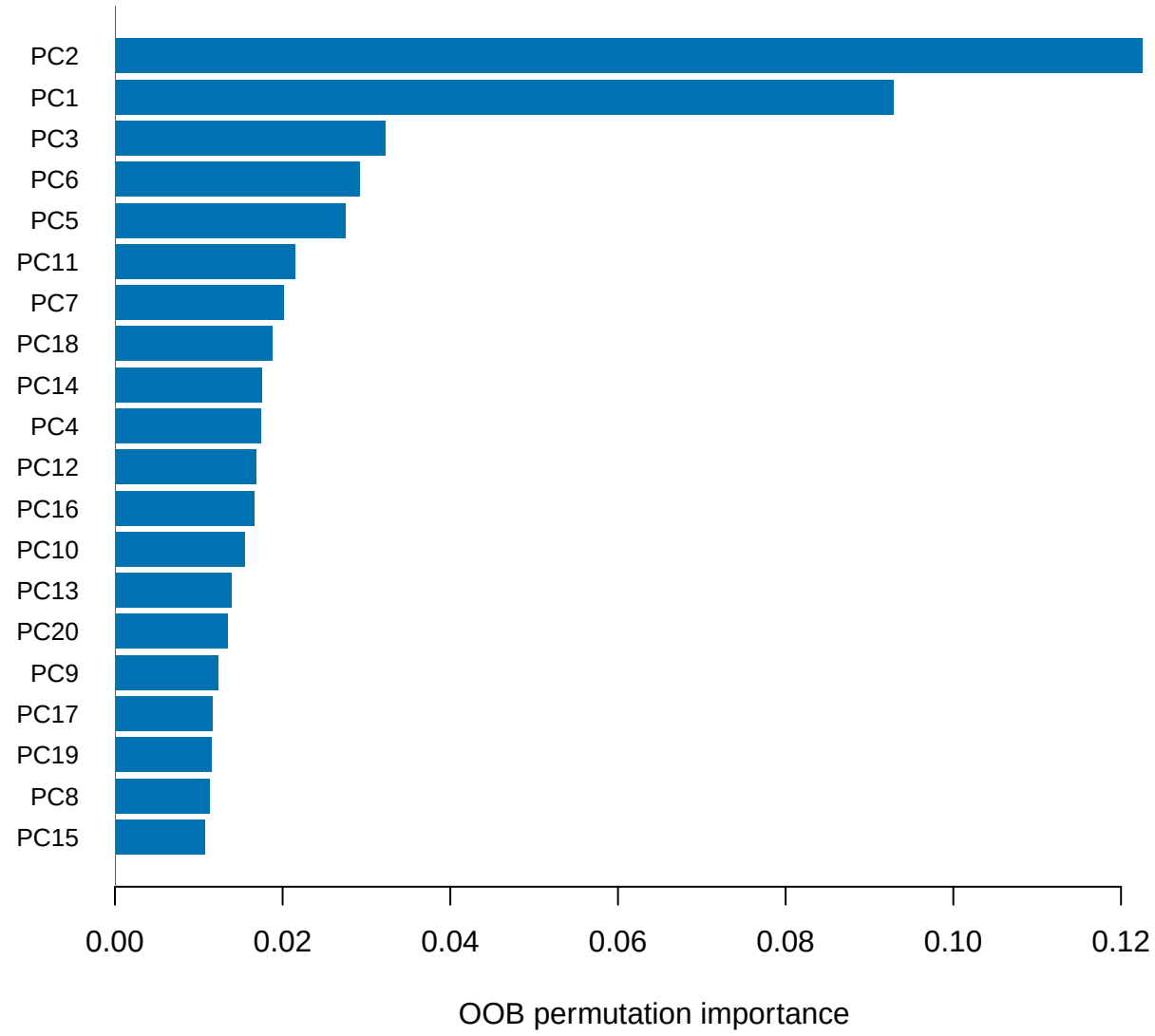


Figure 3: OOB permutation importance for the selected classification forest.

PC2, PC1, PC3 have the highest permutation importance: permuting them harms OOB prediction most. PC2 is more important than PC1, showing that the component explaining the most pixel variance is not necessarily the most useful for classification.

2(c)

The selected model's test misclassification rate is **1.58%** (5 / 317 observations). Test labels were not used for tuning.

```
knitr::kable(
  confusion,
  caption = paste("Test confusion matrix: rows are actual digits;",
                  "columns are predicted digits.")
)
```

Table 1: Test confusion matrix: rows are actual digits; columns are predicted digits.

	1	6	7
1	102	0	2
6	0	106	0
7	0	3	104

All 106 sixes are classified correctly. Two ones are predicted as sevens, and three sevens are predicted as sixes.

3(a)

I would write a concise skill that explains the goal of the random-forest analysis, the main requirements for a valid evaluation, and my preferences for the report. For a capable AI, I would emphasize important requirements, such as preventing data leakage and selecting tuning parameters without using the test set, rather than explaining every step of fitting a random forest. For an AI with less statistical knowledge, I would include more detailed steps and examples. I would also include my preferences, like having code that can be reproduced, clear graphs, and explanations that help make sense of the results. This way, the AI would have enough direction to do the analysis correctly while still being able to adjust its approach depending on the dataset.

3(b)

1. Identify the response variable and predictors, and determine whether the task is regression or classification.
2. Check for missing values, incorrect variable types, and variables that reveal the response or would be unavailable when making predictions.
3. Split the data into training and test sets before fitting the model, and use only the training data for any preprocessing decisions.
4. Set and report a random seed, and record the packages and versions used.
5. Fit a default random forest as a baseline, then tune `mtry` and node size using OOB error or cross-validation on the training data.
6. Evaluate the selected model on the test set only after tuning, using suitable metrics, such as mean squared error for regression or misclassification rate and a confusion matrix for classification.
7. Include permutation variable importance and explain which predictors contribute most to prediction, without treating importance as evidence of causation.
8. Present tuning results in a clearly labeled table, identify the selected setting, and use consistent rounding so comparisons are easy to read.
9. Use simple figures with readable labels and consistent colors, and explain the main finding below each figure.
10. Organize the report with short numbered headings, reproducible R code, and concise explanations in plain language. Focus on what the results mean for the dataset rather than repeating general definitions.